

Advanced Context Management

References

01 O'Reilly, *AI-Native Software Engineering* (early release), "Context Engineering Fundamentals."

02 O'Reilly, *AI-Native Software Engineering* (early release), "Model Context Protocol."

03 N. F. Liu et al., "Lost in the Middle: How Language Models Use Long Contexts," 2023.

04 A. Vaswani et al., "Attention Is All You Need," NeurIPS 2017.

05 Google Cloud, "What is the MCP and how does it work?" cloud.google.com/discover/what-is-model-context-protocol

06 Harada et al., 2024, on instruction count and rule compliance, as cited in the O'Reilly chapter.

07 Model Context Protocol specification. modelcontextprotocol.io

08 Language Server Protocol. microsoft.github.io/language-server-protocol

09 AGENTS.md (agents.md) and Agent Skills (agentskills.io).

What we covered last week

01

The model is the engine

It predicts the next token from patterns. Give it correct facts, then check its work.

02

Context is a budget

The window is finite, and position matters: lost in the middle.

03

An agent is a loop

LLM, tools, context and a loop: reason, act, observe, repeat.

04

The harness makes the agent

Repo context, caching, tools, reduction, memory and subagents.

Today

01

Why context engineering

02

A taxonomy of context components

03

Context assembly and discovery

04

The agent runtime pipeline and session management

05

MCP fundamentals

LAB

Lab 4: a context audit

01 Why context engineering

The same agent, the wrong output

Request: “Add a new API endpoint to our Node.js service.”

01

Used Moment.js

The team stopped using it. No rule said so.

02

Skipped Zod validation

The architecture requires it. It was never in the context.

03

Wrote a unit test

Our automated checks require integration tests. Nobody told the agent.

CAUSE Each mistake traces to missing context, not to a weak model.

Source: O’Reilly, Context Engineering Fundamentals

Prompt versus context engineering

PROMPT ENGINEERING

Phrase one request well

- One input, one interaction
- Improves one request
- “How should I phrase this?”

CONTEXT ENGINEERING

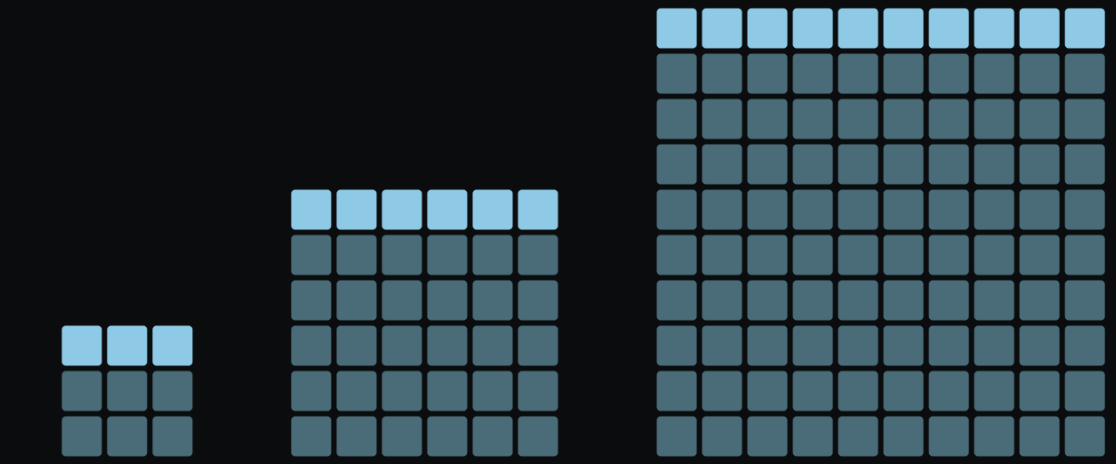
Design everything the model sees

- The whole session, and the whole team
- Manages the whole system
- “Which context gets me the behavior I want?”

KARPATHY “...filling the context window with just the right information for the next step.”

Source: O’Reilly, Context Engineering Fundamentals

The attention problem



3 tokens · 9 pairs 6 tokens · 36 pairs 10 tokens · 100 pairs

THE MECHANISM

Every token looks at every other token

This is how a variable defined at the top of a file is used correctly 300 lines later.

THE COST

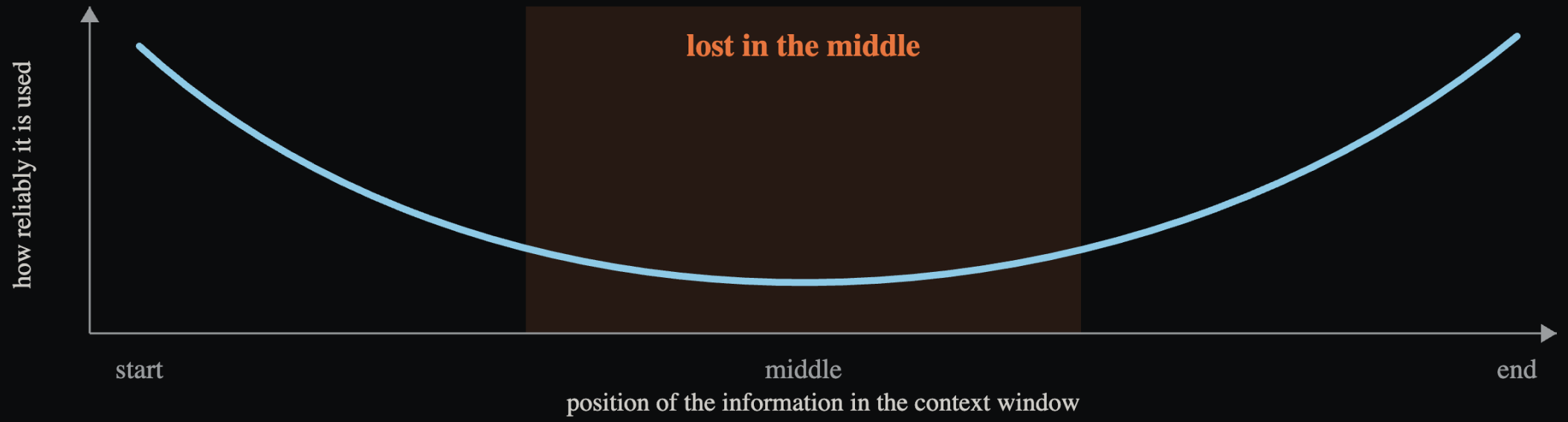
n tokens, n × n pairs

At 100,000 tokens that is 10 billion pairs, and attention is stretched thinner.

RESULT A slow decline, not a sudden drop.

Source: O'Reilly, Context Engineering Fundamentals; Vaswani et al., 2017

The context window is a budget



SIMPLE LOOKUP

Finding one fact: current top models stay reliable across their whole window.

HARDER TASKS

Reasoning over several facts, or resolving contradictions: quality drops well before the window is full.

Context rot



SYMPTOMS

- Repeats the same mistake
- Breaks a rule it agreed to
- Loses the original goal
- Code that fit two steps ago

COUNTERMEASURES

- One session per task
- Reset between tasks
- Keep key facts in files
- Summarize before the window fills
- Restate key rules each session

The human working-memory parallel

HUMANS

About four items at once

Hold ten unrelated facts while solving a hard problem and performance drops. The limit is the resource, not the person.

AGENTS

An attention budget

The budget per item shrinks as the context grows. A full window loses earlier information.

TRY IT · 60 SECONDS

Memorize these ten items while computing 47 × 23, then write them down.

green kettle · 4:20 PM · a parking ticket · smell of bread · Istanbul · 12.8 percent · a loose button · next Monday · olive oil · the word “fragile”

Source: N. Cowan, as cited in the O’Reilly chapter

The right-time problem

IN CONTEXT		THE TASK
Steps for migrating a database	→	Updating a CSS layout
Descriptions of web-scraping tools	→	Refactoring TypeScript types
A detailed skill for adding API endpoints	→	Writing documentation

RIGHT TIME Present but irrelevant context still competes for attention. Load it when the task needs it.

Source: O'Reilly, Context Engineering Fundamentals

Two failure modes

TOO LITTLE CONTEXT

The agent fills the gaps

- It falls back on common patterns from training
- Fine in the abstract, wrong for your system
- The Node.js endpoint is an example

TOO MUCH CONTEXT

Noise competes for attention

- Critical lines get buried in the middle
- Slower, costlier and less accurate
- Every token costs money and time

GOAL Assemble enough context for the model to reason well, structured so that attention lands where it should.

Source: O'Reilly, Context Engineering Fundamentals

Mandatory and Optional Context Components

Only the user's query is mandatory. Everything else is a deliberate choice with a cost.

QUESTION 1

How often does it improve the output?

QUESTION 2

What would break if it were absent for a session?

RULE Rarely useful and nothing obvious breaks? Load it on demand instead of keeping it always on.

Source: O'Reilly, Context Engineering Fundamentals

02 A taxonomy of context components

The components at a glance

- **System prompt**
- **Tool descriptions**
- **Rules and conventions**
- **Environment context (AGENTS.md)**
- **List of skills (names only)**
- **Conversation history, tool results**
- **Your message and @-referenced files**

Blue: present from the start. Grey-blue: grows during the session.

ALWAYS ON

Loaded before you type

Costs tokens in every session, so it must be short and relevant.

ON DEMAND

Loaded when the task needs it

Full skill text, referenced files and retrieved data stay out until needed.

CUSTOM AGENTS

Same parts, packaged under a name

A role bundles instructions, a chosen model and a restricted tool set.

System prompts: precision over volume

A SYSTEM PROMPT USUALLY HAS

Role and goals
Who the agent is and what it is for.

Rules
What it must always and never do.

Output format
Structure and level of detail.

THE "CURSE OF INSTRUCTIONS"

If each rule is followed 95% of the time, the chance that all rules are followed is 0.95 raised to the number of rules.

5 rules	10 rules	20 rules
77%	60%	36%
all followed	all followed	all followed

TIP Write rules as testable statements: when [trigger], the system shall [action].

Source: Harada et al., 2024, as cited in the O'Reilly chapter. Assumes 95% per rule.

User input and user-provided context

THE REQUEST

Sets the task and its boundary

A question, a change request, a spec, a bug or a refactor. It decides which other components matter.

@ REFERENCES

Pull in exactly the slice you need

Files, functions or docs you point to in your message enter the context when the task starts.

SIGNAL Retyping the same context for a recurring task? Move it into a rule or a skill.

Source: O'Reilly, Context Engineering Fundamentals

Rules, style guides and constraints

ALWAYS ON

Loaded into every session

- Project conventions and coding standards
- Design and security limits
- Behavioral guardrails

A GOOD RULE

A rule with its reason

“Use date-fns instead of moment.js, because Moment is deprecated and adds 70 KB to the bundle.”

SCOPE Rules form a hierarchy: global preferences, project conventions, then file-specific rules.

Source: O’Reilly, Context Engineering Fundamentals

What are Agent Skills?

An open standard for adding abilities to agents. A skill is a reusable package, saved with your code, that teaches an agent how to do one kind of task.

AGENTS.MD

Always on

Project rules every agent reads in every session: commands, style, conventions.

SKILL

Loaded when needed

Know-how for one task, like a code review or a release, pulled in only when the task calls for it.

Sources: agentskills.io · Cursor docs

How skills work

1 **Discover**	At startup the agent finds the skills in its skill folders.	`loads: name + description`
2 **Decide**	It picks a skill when the task matches its description.	`loads: full SKILL.md`
3 **Use**	It opens scripts, references or assets only when a step needs them.	`loads: files on demand`

Or call one yourself: type / in the chat and pick the skill.

Sources: agentskills.io · Cursor docs

Tools

A CONTEXT COMPONENT

Descriptions live in the window

- Name, parameters, description
- Cost tokens on every request
- More tools, higher baseline cost

A WAY TO FETCH CONTEXT

The agent pulls in more context

- File operations and codebase search
- Shell, web access and task tracking
- Concise outputs keep the window lean

RULE Give a task only the tools it needs. If a human could not choose between two tools, neither can the agent.

Source: O'Reilly, Context Engineering Fundamentals

The model only sees the description

```
{
  "name": "read_file",
  "description": "Read the full text of a
    file. Use for source code, config or
    docs. Do not use for binary files or
    files larger than 1MB.",
  "parameters": {
    "path": { "type": "string",
      "description": "Absolute path" }
  },
  "required": ["path"]
}
```

VAGUE

"reads a file"

When? Instead of which tool?

CLEAR

What it does, when to use it, and when *not* to.

ANTHROPIC'S SWE-BENCH AGENT

More time went into tools than the prompt. Requiring absolute paths removed a whole type of error.

What is AGENTS.md?

A README for agents: one plain Markdown file in your repo that tells any coding agent how to work on your project.

Open format

Maintained by the Agentic AI Foundation, part of the Linux Foundation

20+ agents

Codex, Cursor, Copilot, Gemini CLI, Jules, Aider, Zed and more

60k+ projects

Open-source repos already ship one

Source: agents.md

Conversation history and memory

INSIDE THE WINDOW

In-context memory

The conversation history: messages, tool calls and results. Gone when the session ends, and summarized when the window fills.

OUTSIDE THE WINDOW

Long-term memory through files

PROGRESS.md, spec.md and tasks.md. They survive restarts, people can read them, and Git can track them.

OUTSIDE THE WINDOW

Retrieved memory with RAG

Knowledge is stored in a searchable index. The relevant pieces are found and added to the context.

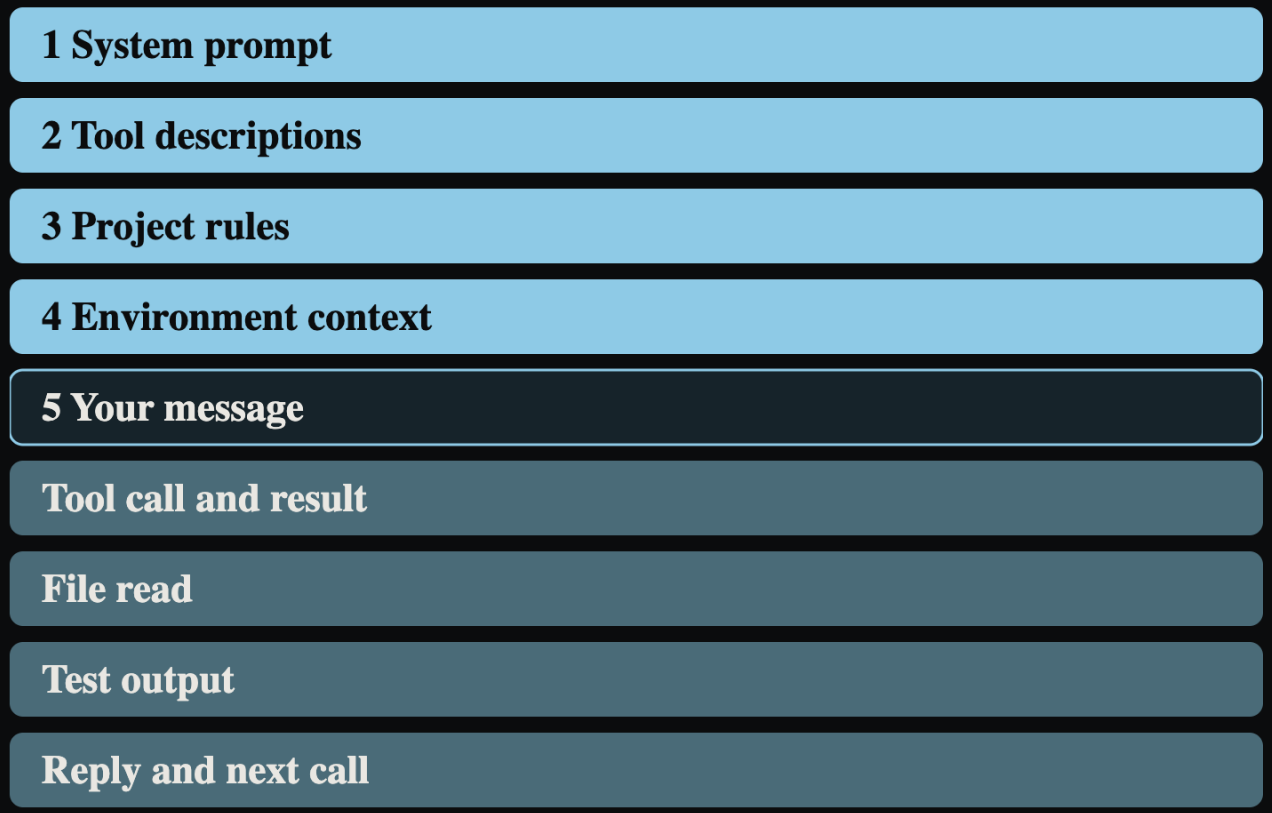
OUTSIDE THE WINDOW

Agent-generated memory

Notes the agent writes for itself, such as a handoff note it reads at the start of the next session.

03 Context assembly and discovery

The context assembly process



Top: the fixed part. Bottom: content added during the session.

FIXED PART

Set before the first message

The same in every session of a project.

ADDED DURING THE SESSION

Added as the agent works

Files read, test output, tool results and replies.

COST 5,000 tokens of setup leave 5,000 fewer tokens for real work.

Source: O'Reilly, Context Engineering Fundamentals, Figure 3-1 (redrawn)

When an agent ignores a constraint

1	**Was it loaded?**	Check that the rule, file or skill actually reached the session.
2	**Where did it sit?**	The start and end of the window get more attention than the middle.
3	**Did it stand out?**	One line in a long rules file may as well be invisible.
4	**Was it contradicted?**	Another rule, a tool description or an earlier turn may say the opposite.
5	**Has it been lost?**	In a long session it may have been summarized away.

Source: O'Reilly, Context Engineering Fundamentals

Autonomous context discovery

How agents explore before they write

WHAT YOU PROVIDE

Context you prepare

- Rules you write
- Files you reference
- System prompts you set up

WHAT THE AGENT FINDS

Context it discovers

- Information somewhere in the codebase
- Not in a rule file or preloaded document
- Found before the first line is written

WHY Some tasks need information that no rule file contains. The agent has to go and find it.

Source: O'Reilly, Context Engineering Fundamentals

Code indexing and semantic search

HOW IT WORKS

A searchable index of the repo

- Your request is matched by meaning
- The closest files and functions load automatically
- All before the agent starts reasoning

WHAT IT FINDS

Related code you did not name

- A utility in a module you did not reference
- An existing test pattern to follow
- The interface the new code must implement

WHERE IT FAILS

A snapshot of the code

- New files may not be indexed yet
- Design rules may not match your wording
- It cannot know what it does not know

Source: O'Reilly, Context Engineering Fundamentals

CLI-based exploration

`grep -r "verifyToken" src/`	Every place the login check is used
`grep -r "new Pool" .`	Where the database connection is created
`git log --oneline -20`	What changed recently, to orient the search

BEST FOR

Structural questions

Where is this called, which files import this module, what changed this week. More precise than embedding search.

THE COST

Every result stays in the window

Grep output and directory listings accumulate. Ten exploration calls can use a meaningful share of the budget.

Source: O'Reilly, Context Engineering Fundamentals

Language servers

Symbolic code understanding

GO TO DEFINITION

“Where is this function defined?”

FIND REFERENCES

“What references this interface?”

DIAGNOSTICS

“What type errors exist in this file right now?”

WHAT YOU GAIN

A live view of the code’s structure

Answers reflect your code as it is now, including unsaved changes. Safer renames and restructuring than search results.

WHAT IT COSTS

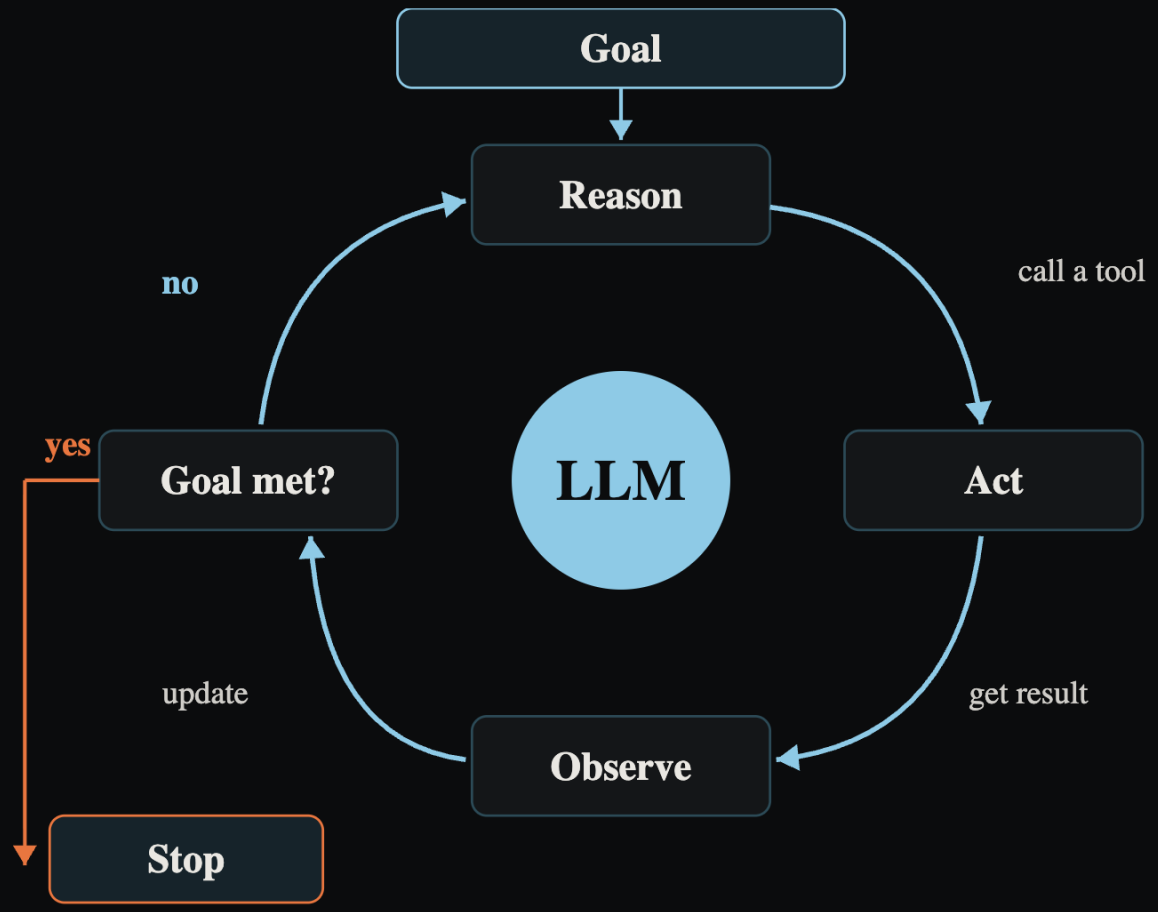
Setup, and limits of code analysis

Needs a server per language. Code that is loaded or generated while the program runs can still confuse it.

Source: O’Reilly, Context Engineering Fundamentals

04 The agent runtime pipeline and session management

The iterative loop



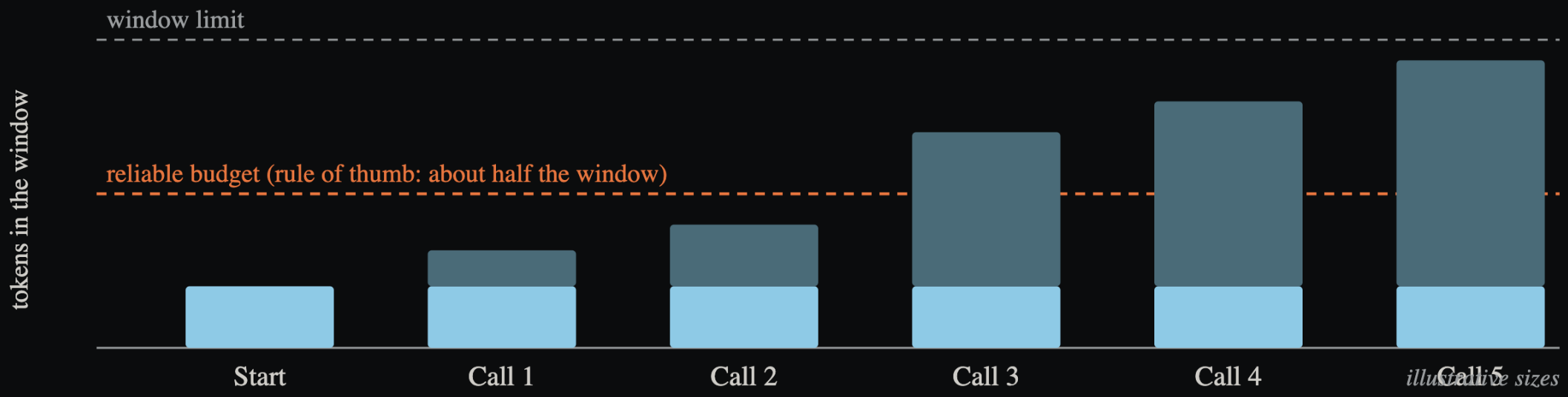
SAME LOOP, MANY NAMES

- Reason–Act–Observe
- Plan–Execute–Reflect
- Think–Do–See

IT ENDS WHEN

- The goal is met
- It can't resolve something
- It hits a budget or step limit

Context cost compounds with every call



Results stay in the window

A verbose result costs budget for the rest of the session.

Design tools to return less

The relevant table, not the whole database layout. A page of results, not all of them.

Source: O'Reilly, Context Engineering Fundamentals, Figure 3-2 (redrawn, illustrative)

Session management: when to reset

SIGNAL 1

Old mistakes return

The agent breaks a constraint it handled correctly earlier.

SIGNAL 2

Answers get vague

It hedges about facts it should be confident about.

SIGNAL 3

The window is nearly full

Many tool calls, and the tool is about to summarize older content.

WHY A new session starts clean: no stale tool results, and nothing buried in the middle.

Source: O'Reilly, Context Engineering Fundamentals

The handoff artifact

1 · END OF SESSION

Ask for a handoff file

Before closing, have the agent summarize the session into a file.

2 · THE FILE

Three short lists

```
PROGRESS.md
Decided: …
Done: …
Next: …
```

3 · NEW SESSION

Start from the file

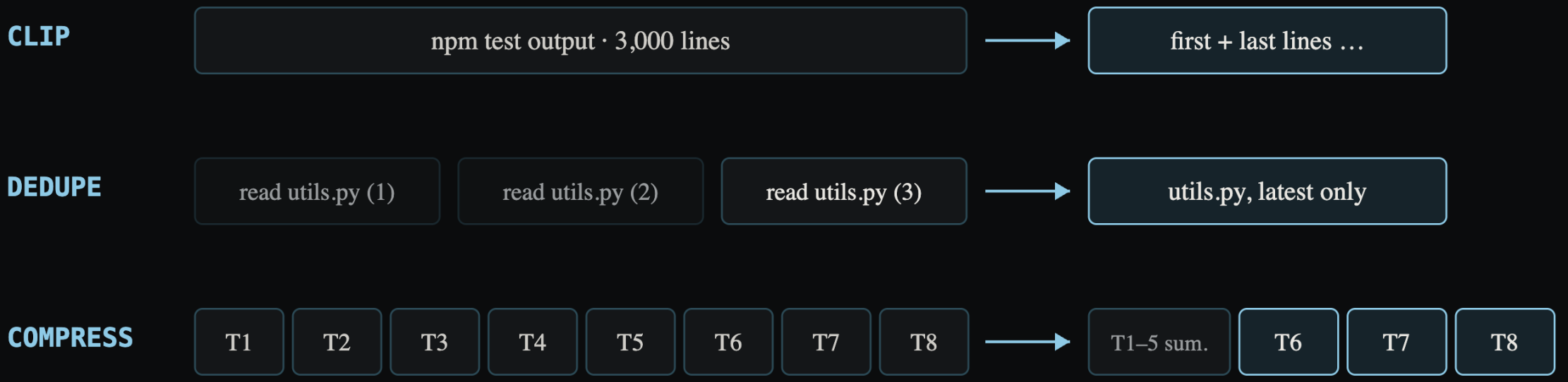
“Read PROGRESS.md and continue from where we left off.”

AUTO-SUMMARY Automatic summaries keep the thread but lose detail. Save decisions that shape later work in a file.

Source: O’Reilly, Context Engineering Fundamentals

RECAP · COMPONENT 04

Context reduction



> “A lot of apparent ‘model quality’ is really context quality.”
>
> Sebastian Raschka, Components of A Coding Agent

05 MCP fundamentals

What is the MCP and how does it work?

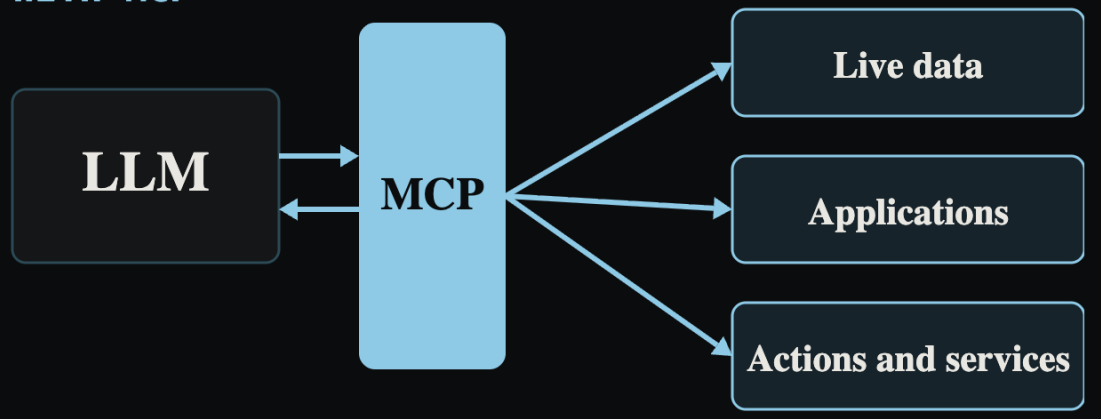
An open standard that gives LLMs a secure, standardized way to reach external data, applications and services.

WITHOUT MCP

LLM

- Knowledge frozen at training time
- No access to live data
- Cannot take actions

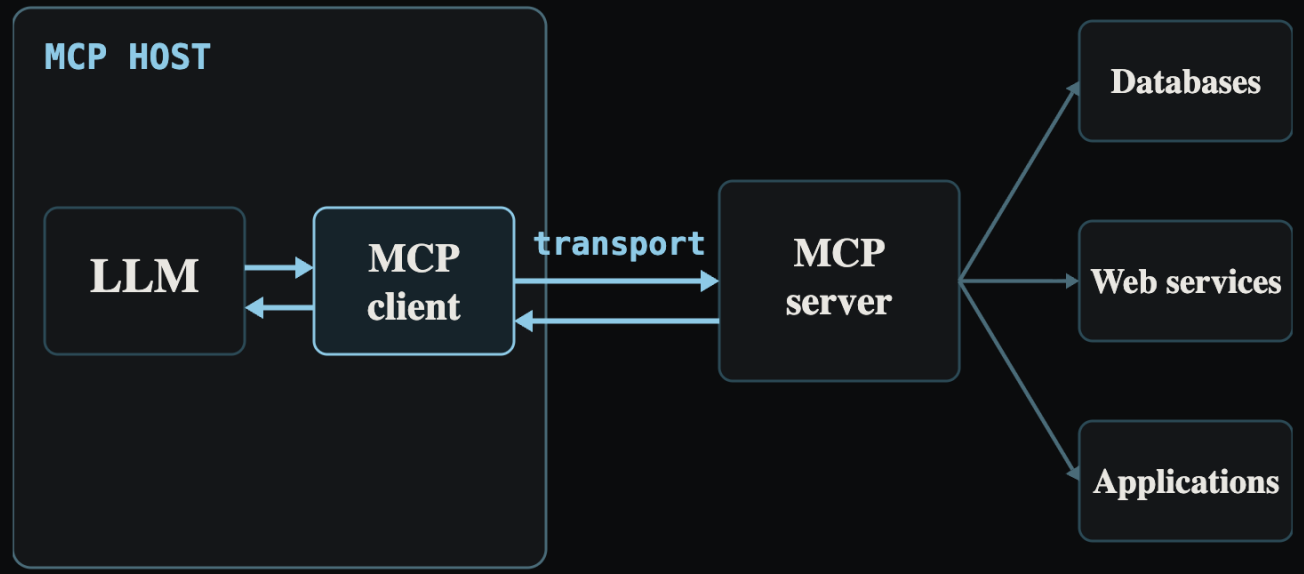
WITH MCP



ORIGIN Introduced by Anthropic in November 2024. It builds on tool use and function calling.

Source: Google Cloud, What is the MCP and how does it work?

MCP architecture and components



The host contains the LLM. The server reaches the outside world.

MCP HOST

The AI app that contains the LLM, such as an AI-powered code editor. Where the user interacts.

MCP CLIENT

Sits inside the host. Translates between the LLM and the server, and finds available servers.

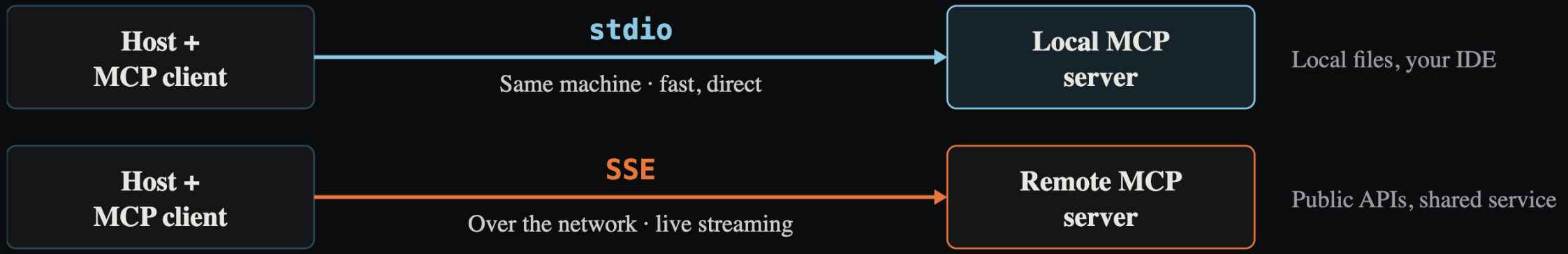
MCP SERVER

The external service that supplies data or capabilities, from databases to web services.

Source: Google Cloud, What is the MCP and how does it work?

Transport layer

Messages in a standard format (JSON-RPC 2.0) travel between client and server in one of two ways.



Standard input/output (stdio)

For local resources. Fast, and each request waits for its reply.

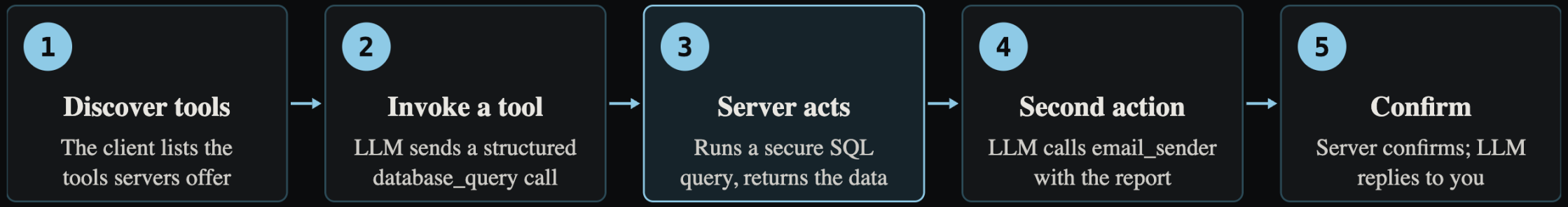
Server-sent events (SSE)

For remote resources. Efficient, live streaming of data.

Source: Google Cloud, What is the MCP and how does it work?

How does the MCP work?

Request: “Find the latest sales report in our database and email it to my manager.”



TOOLS FOUND

database_query · email_sender

KEY IDEA

The LLM asks. The client routes. The server acts.

Source: Google Cloud, What is the MCP and how does it work?

Benefits of using the MCP

01

Reduced hallucinations

LLMs predict from training data and can invent facts. MCP gives them a clear route to reliable external sources, so answers are more truthful.

02

Increased utility and automation

Ready-made tools for business software, content stores and development tools let LLMs update customer records, look up news or run calculations on their own.

03

Easier connections

Before MCP, every model-and-tool pair needed its own connection: the N × M problem. One open standard lowers cost and makes switching providers easier.

ANALOGY Like a USB-C port: one standard connector instead of a custom cable for every device.

Source: Google Cloud, What is the MCP and how does it work?

MCP and security

User consent and control

Users understand and approve every action and data access, ideally through clear authorization screens.

Data privacy

Ask permission before exposing user data to servers, and protect sensitive data with access controls and encryption.

Tool safety

Tools can run code. Do not trust tool descriptions from unverified servers; ask permission before a tool runs.

Secure output handling

Clean inputs and filter outputs to stop web attacks such as cross-site scripting (XSS), and keep sensitive data out of prompts.

Supply chain security

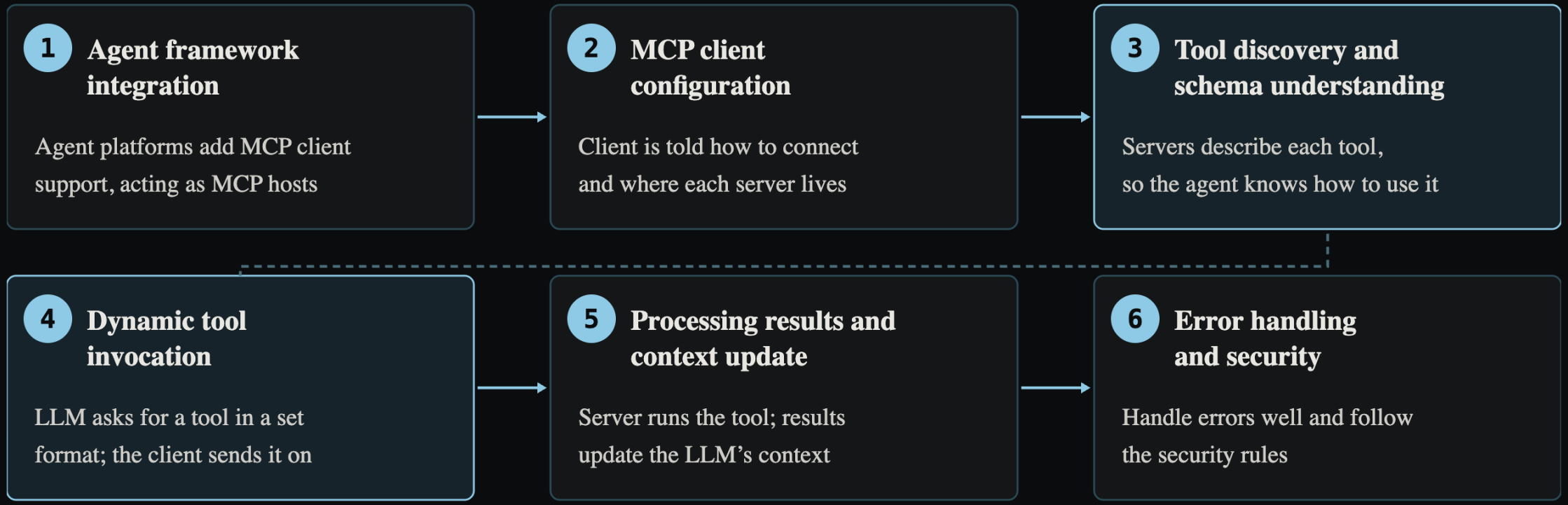
Every server and tool in the chain must be reliable, to avoid breaches, failures or biased results.

Monitoring and auditing

Log activity and tool use to spot unusual behavior and to support incident response.

Source: Google Cloud, What is the MCP and how does it work?

Connecting MCP servers with AI agents



Source: Google Cloud, What is the MCP and how does it work?

Questions?

Next week:
Agentic Codebase Structure

NEXT WEEK: THE REPOSITORY



What belongs in the repo so the agent starts well?